

SOFIA ML REGRESSION 2026 SUMMER

Predicting a Linear Target from 20 Anonymized Features

Approach, Experiments & Final Results

Huimei Jiang

Kaggle: Sofia ML Regression 2026 Summer | Public $R^2 = 0.57124$ · Private $R^2 = 0.54018$

TL;DR

Linear signal hides in 14 of 20 features

The task looked like a generic regression problem, but everything in the data — model comparisons, feature correlations, and interaction tests — pointed to one story: a clean linear relationship buried in 20 features, 6 of which are pure noise.

- 1 Impute:** Median-fill the ~1% missing values per feature
- 2 Select:** Keep the 14 features with real (bootstrap-stable) signal
- 3 Regularize:** RidgeCV chooses the penalty strength automatically
- 4 Hedge:** Average predictions across the 13/14/15-feature cut

PUBLIC R^2 / PRIVATE R^2

0.571 / 0.540

R^2 · coefficient of determination

Public R^2 (v3)	0.57124
Private R^2 (final)	0.54018
Best CV R^2 estimate	0.5387
CV vs. private gap	only 0.0015

A clean, anonymized, near-orthogonal dataset

2,500 / 2,500

Train / test rows

20

Anonymized features (x0–x19)

~1%

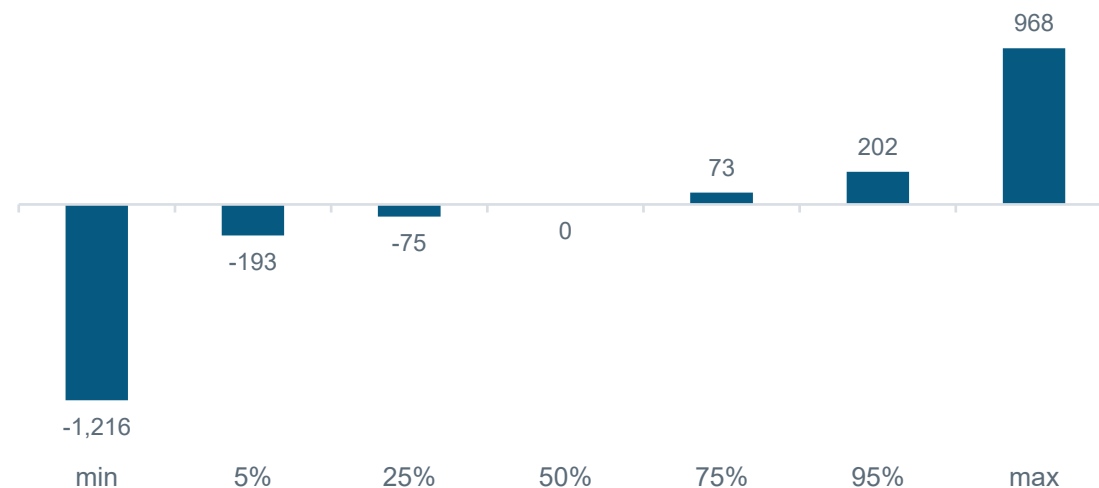
Missing values per feature

≈ 0

Max pairwise feature correlation

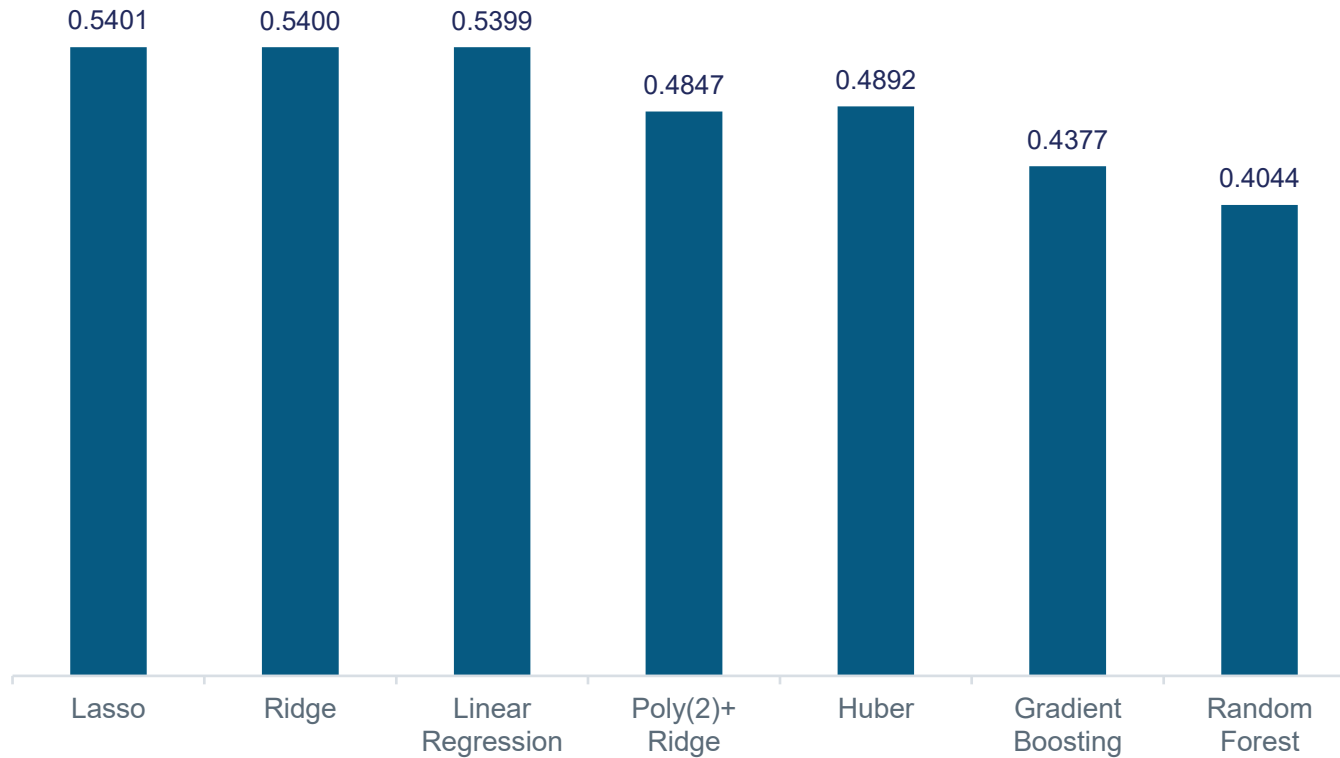
target is heavy-tailed: mean ≈ 1.8 , std ≈ 149 , range $-1,215$ to 968 — consistent with a linear combination of ~ 14 features rather than a small set of outliers.

Target distribution (quantiles)



Missing data: scattered evenly across all 20 columns (no single problematic feature) → handled with simple median imputation.

Linear models beat tree ensembles decisively

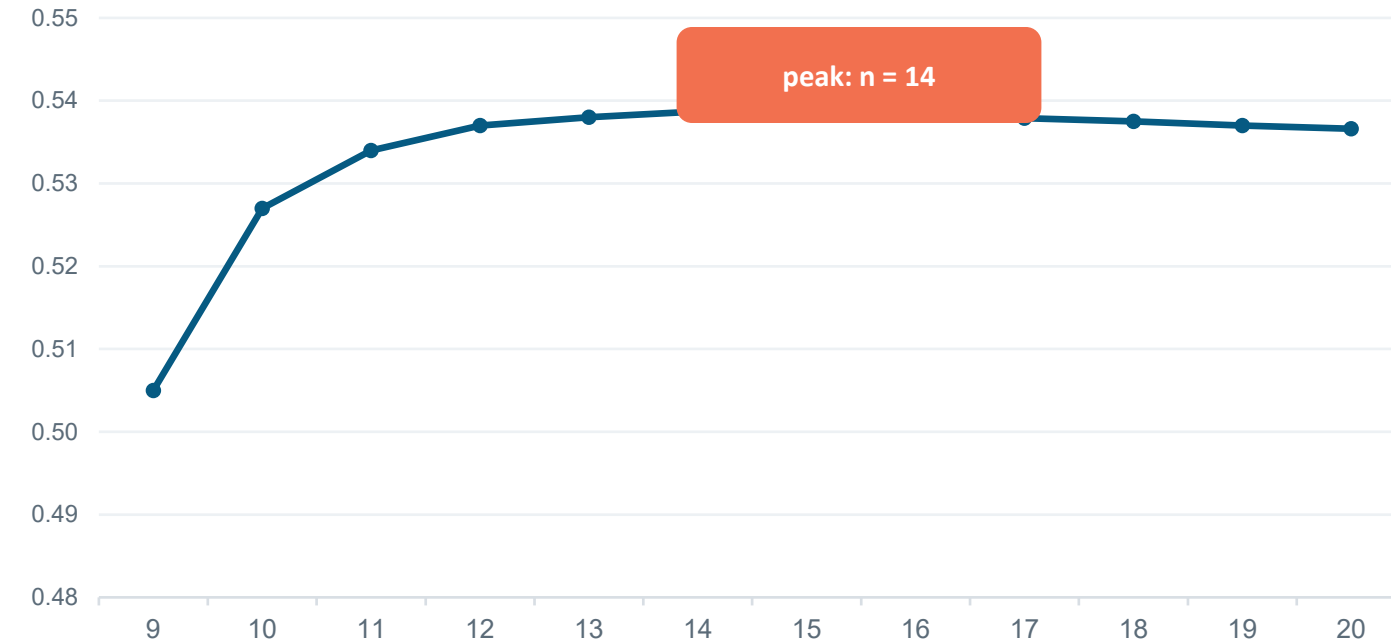


5-fold cross-validated R^2 on training data

Why linear wins

- All linear variants (Lasso / Ridge / plain OLS) land within 0.0002 of each other — the ceiling is set by the data, not the regularizer.
- Adding degree-2 interaction terms hurt performance — confirms there's no hidden nonlinearity to capture.
- Random Forest & Gradient Boosting underfit the linear structure and overfit the noise instead.

14 of 20 features carry the real signal



Number of features retained (ranked by |coefficient|)

Confirmed by bootstrap

200 bootstrap resamples of RidgeCV coefficients show a clean split:

14 features: |coef| 5–58, std well below the mean

6 features: |coef| ~1.6–2.9, indistinguishable from zero noise

Interaction terms among the top features made CV R^2 worse, ruling out hidden nonlinearity.

Three submissions, three small, evidence-based steps

Version	Approach	CV R ²	Public R ²
v1	All 20 features + RidgeCV	0.5366	0.57085
v2	Top-14 features (by coef) + RidgeCV	0.5387	0.57092
v3	Avg. of 13 / 14 / 15-feature RidgeCV models	0.5385	0.57124

Each version was a direct response to what the previous one revealed

- v1 → v2 motivation:** Ridge coefficients on all 20 features showed a clean split — 14 features with real signal, 6 statistically indistinguishable from zero. Dropping the 6 noise features nudged CV R² up.
- v2 → v3 motivation:** the exact 14-feature cutoff was itself somewhat arbitrary (13, 14, and 15 all scored within noise of each other). Averaging across that uncertainty, rather than committing to one cutoff, gave a small additional gain.

Every experiment tested a specific hypothesis

WORKED

● Dropping 6 noise features

Bootstrap showed their $|coefficients|$ were the same size as their own sampling noise — keeping them just added variance, not signal.

● RidgeCV's automatic penalty

With near-zero feature correlation, its main job was gently damping the noisy features rather than fixing multicollinearity.

● Averaging the 13/14/15 cut

The exact boundary between signal and noise was itself uncertain — hedging across nearby cutoffs reduced that selection risk.

● Trusting CV over the public score

CV (0.5387) landed 0.0015 from the true private result (0.540); the public split alone was noisier and less trustworthy.

DIDN'T WORK

● Interaction / polynomial terms

CV R^2 got worse when added — confirms the relationship is genuinely linear, with no hidden curvature to capture.

● Robust regression (Huber)

Underperformed plain Ridge — there were no real outliers to protect against, so it just down-weighted real signal.

● Automatic selection (Lasso/ElasticNet)

Its sparsity penalty shrank useful coefficients along with noisy ones, instead of making a clean cut like manual selection.

● Bagging / model ensembling

The base model was already low-variance (linear, few features) — little variance was left for bagging to reduce.

FINAL RESULTS

Public and private scores, side by side

The competition rules cautioned against overfitting to the public leaderboard. Our public score (0.57124) and final private score (0.54018) differ by about 0.03 — worth understanding rather than reading as a failure.

0.571

Public R^2
(48% of test)

0.540

Private R^2
(52% of test, final)

0.539

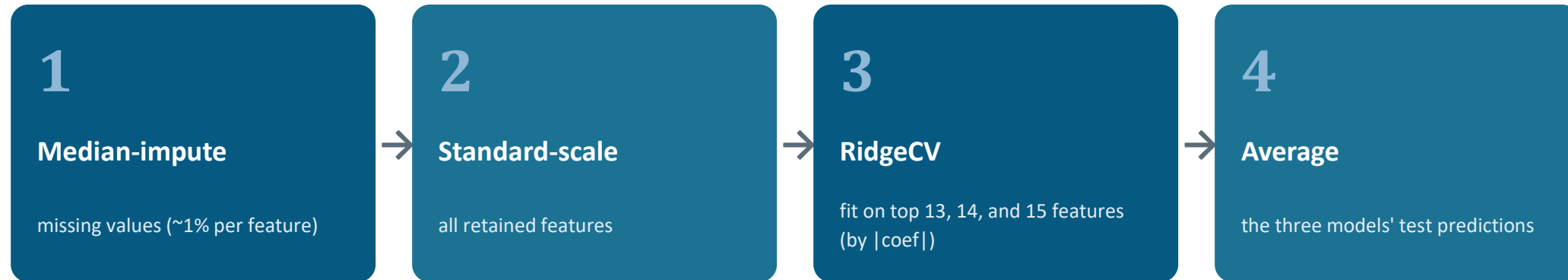
Our CV estimate
(train-only)

Key insight: our cross-validated estimate (0.539) landed just 0.0015 from the true private score — far closer than the public leaderboard (which overstated performance by ~ 0.03). Trusting CV over the public score, as we did throughout, was the right call.

Why CV predicted this so well

- The public leaderboard uses only 48% of the test set — small enough that its own sampling noise can swing scores by hundredths.
- Our internal CV averaged over 20 repeats of 5-fold splits — far more resamples than the public/private split alone, cancelling out that noise.
- Because features and regularization were chosen using CV, not tuned to the public number, performance carried over consistently to the private set.

Final pipeline: median impute → scale → RidgeCV



Reproducible code

GitHub repo: <https://github.com/HJ313619/ml-hw-summer-2026-challenge>

Contains: `solve_challenge_v3.py` (final, best submission)

Evaluation metric: R^2 (coefficient of determination), matching scikit-learn's `model.score(testX, testY)`